

Thuật toán CART là gì?



Bước 1

Phân chia tập huấn luyện thành 2 tập con sử dụng 1 đặc trưng k và ngưỡng t_k .



Bước 2

Chọn k và t_k sao cho tạo ra 2 tập con thuần khiết nhất (Gini Impurity nhỏ nhất).



Bước 3

Lặp lại đệ quy quá trình trên với từng tập con.



Bước 4

Dừng lại khi đạt đến max_depth hoặc không thể giảm thêm độ vẩn đục.

Hàm Chi phí của CART (Phân loại)

$$J(k, t_k) = \frac{m_{\text{left}}}{m} G_{\text{left}} + \frac{m_{\text{right}}}{m} G_{\text{right}}$$

$G_{\text{left}}, G_{\text{right}}$: Độ vẩn đục Gini của tập con bên trái/phải.

$m_{\text{left}}, m_{\text{right}}$: Số lượng mẫu của tập con.

m : Tổng số lượng mẫu tại nút đang xét.



Thuật toán Tham lam (Greedy Algorithm):

Chỉ tìm kiếm mức phân tách tốt nhất tại hiện tại, không nhìn về tương lai. **Không bảo đảm tối ưu cục bộ.**

Gini Impurity hay Information Entropy?

Khái niệm

- Mặc định, CART dùng Gini Impurity (criterion="gini"). Có thể đổi sang dùng Entropy (criterion="entropy").
- Trạng thái thuần khiết: Entropy = 0 khi tập hợp hoàn toàn thuần khiết (chỉ chứa 1 lớp).



Công thức Shannon's Entropy:

$$H_i = - \sum_{k=1, p_{i,k} \neq 0}^n p_{i,k} \log_2(p_{i,k})$$

So sánh: Gini vs. Entropy



Gini Impurity (Lựa chọn Mặc định tốt)

- Tính toán nhanh hơn một chút (không chứa phép Logarit).
- Xu hướng: Thường cắt ra nhánh lớn nhất.



Information Entropy

- Tính toán chậm hơn.
- Xu hướng: Tạo ra các cây cân bằng hơn (balanced trees).

Kết luận: Trong hầu hết mọi trường hợp, Gini và Entropy tạo ra những cây rất giống nhau.

Độ phức tạp Thời gian (Time Complexity)



Quá trình **Suy luận**
(Dự đoán) - **Cực Nhanh**



- Phải duyệt từ gốc đến lá.
- Độ sâu cây: $O(\log_2(m))$
- Phức tạp dự đoán: $O(\log_2(m))$
(Độc lập với số lượng đặc trưng!)



Quá trình **Huấn luyện** -
Rất Chậm (với dữ liệu lớn)

- Thuật toán so sánh mọi đặc trưng trên mọi ngưỡng.
- Phức tạp thời gian:
 $O(n \times m \log_2(m))$
- **Rất chậm** đối với các tập dữ liệu có m lớn (Hàng triệu dòng).



Tại sao phải Chính quy hóa?



Bản chất (Mô hình Non-parametric):
Số lượng tham số không được ấn định trước.
Mô hình phát triển tự do theo dữ liệu.



Hậu quả (Không có giới hạn):
Cây cổ phân nhánh đến khi mỗi chiếc lá
chỉ có 1 điểm duy nhất.



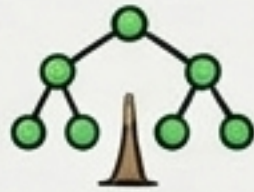
Thực tế:
Tương đương việc ghi nhớ học vẹt bộ dữ liệu.



Kết quả cuối cùng:
Gây ra hiện tượng Overfitting cực kỳ nghiêm trọng.

Các Siêu tham số Chính quy hóa

max_depth



Độ sâu tối đa của cây (Default: None).



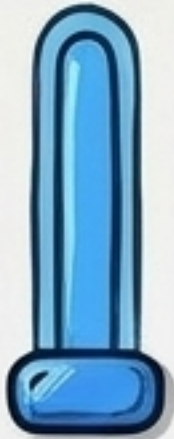
← Giảm để hạn chế Overfitting.

min_samples_split



Số lượng mẫu tối thiểu một nút phải có trước khi được phân chia.

min_samples_leaf



Số mẫu tối thiểu một nút lá phải có.

max_features



Số lượng đặc trưng tối đa được xem xét tại mỗi lần chia.

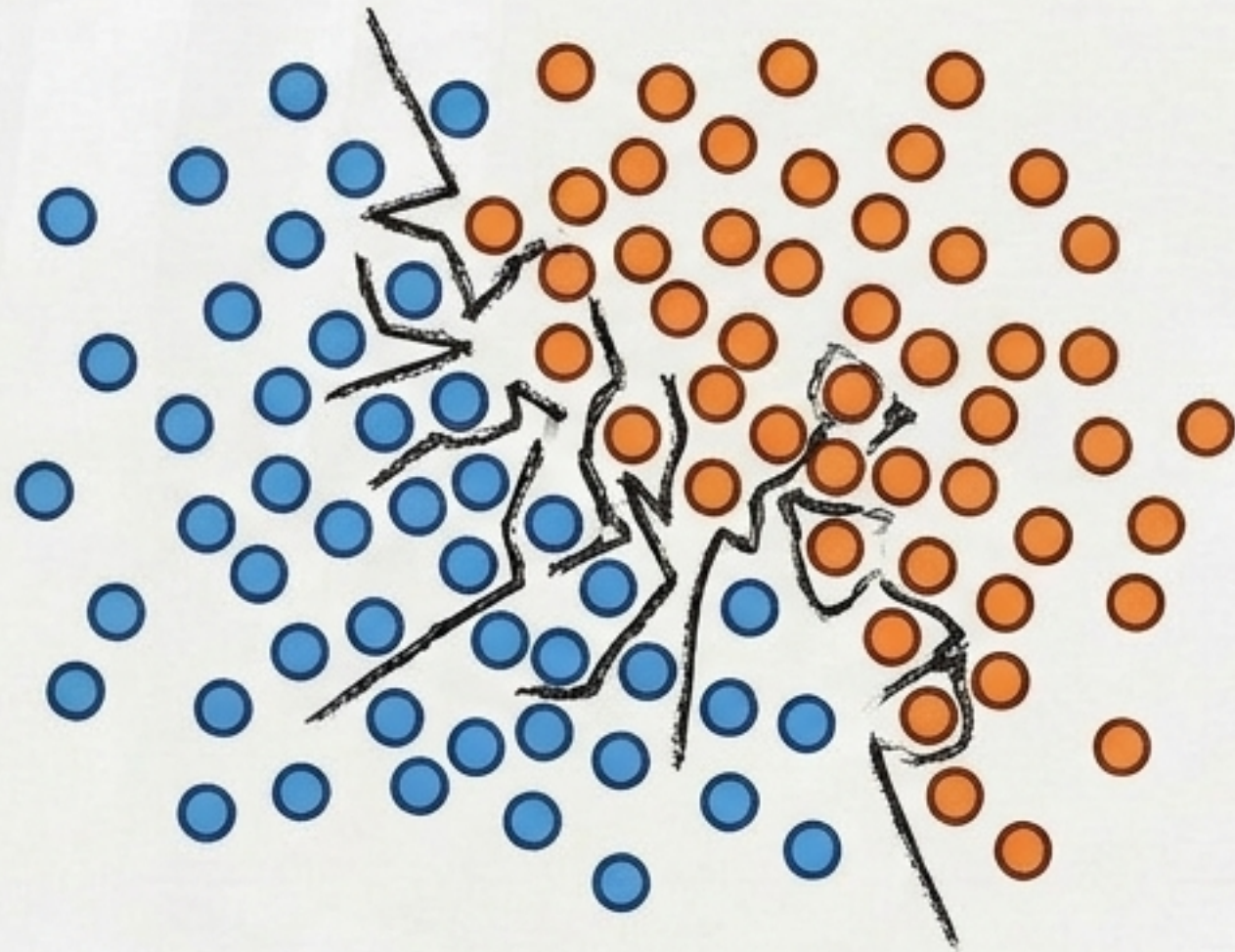
max_leaf_nodes



Số lượng nút lá tối đa.

Hiệu ứng của Chính quy hóa

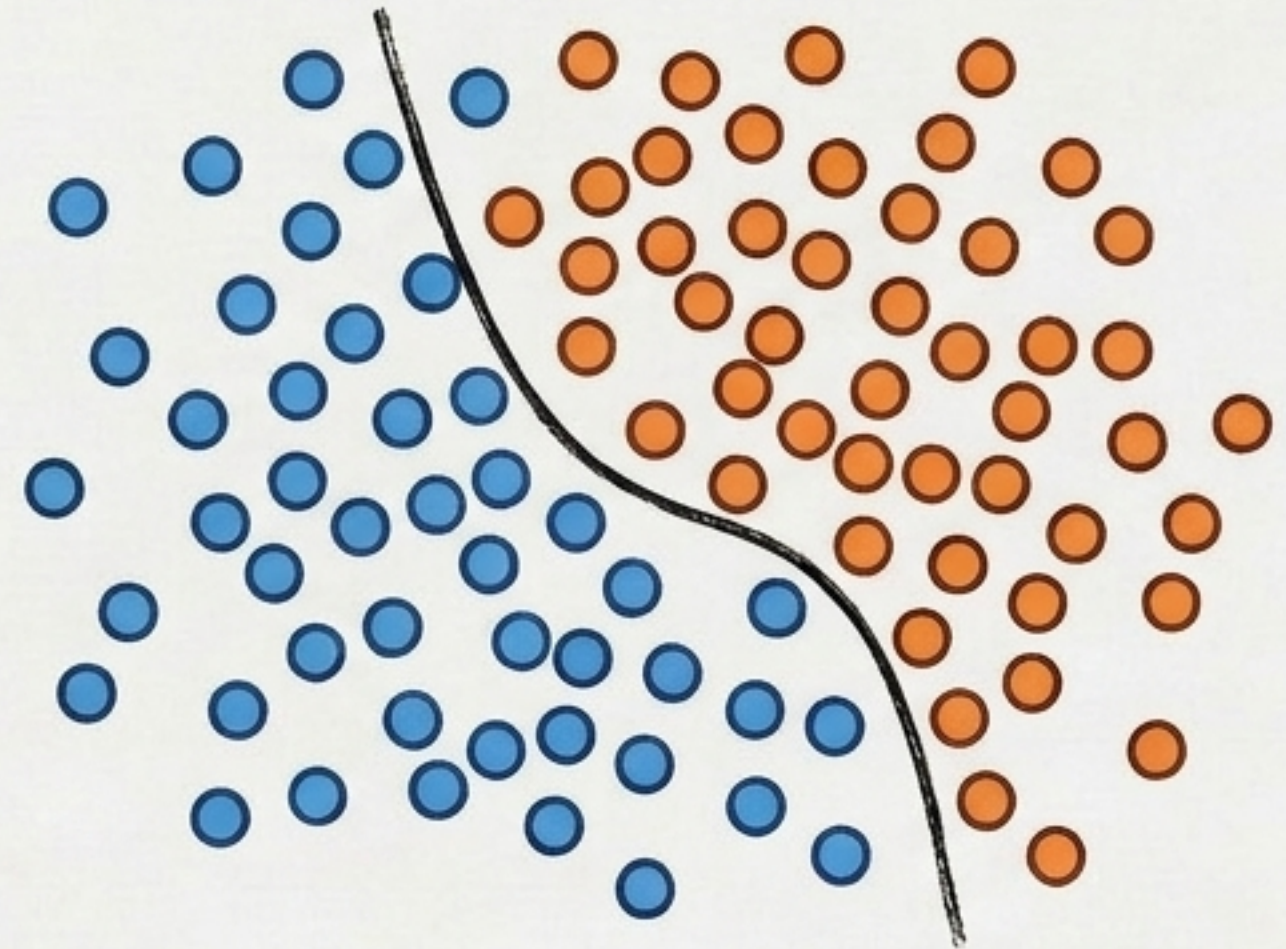
Cây không giới hạn



$\text{min_samples_leaf}=1$

→ Ranh giới ngoằn ngoèo, Overfitting nghiêm trọng.

Cây bị ràng buộc



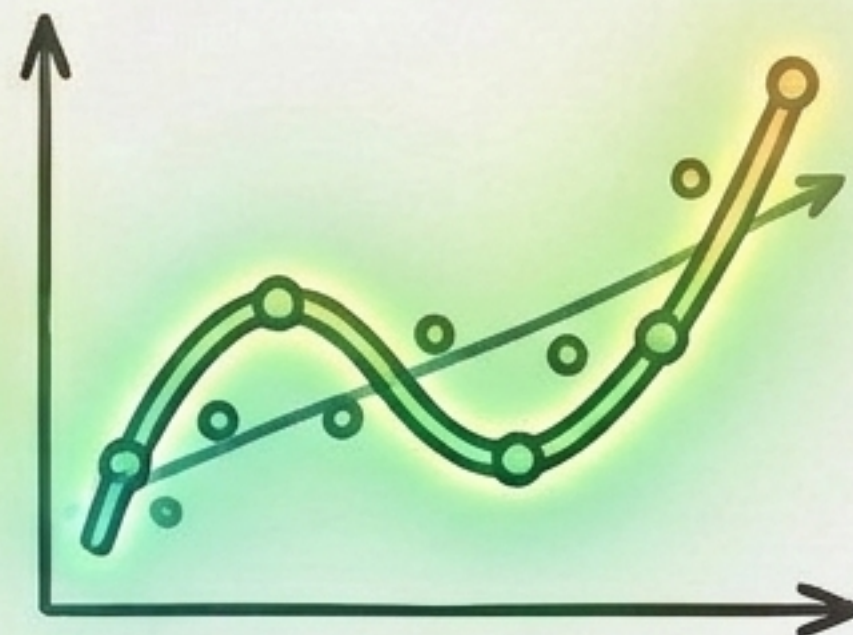
$\text{min_samples_leaf}=4$

→ Ranh giới mượt mà hơn, khả năng tổng quát hóa (Generalization) tốt hơn trên dữ liệu mới.

Hình 6-3: Cây không chính quy (trái) vs Chính quy hóa (phải).

Lớp DecisionTreeRegressor (Hồi quy)

Cây quyết định cũng giải quyết xuất sắc các bài toán Hồi quy.



Sự khác biệt
cốt lõi:

- Thay vì dự đoán một nhãn lớp...
- Nút lá dự đoán một giá trị liên tục (Trung bình cộng của tất cả các mẫu nằm trong lá đó).

Hàm mục tiêu:

Sử dụng MSE (Mean Squared Error) để tối ưu.

Code Python: Hồi quy dữ liệu Quadratic



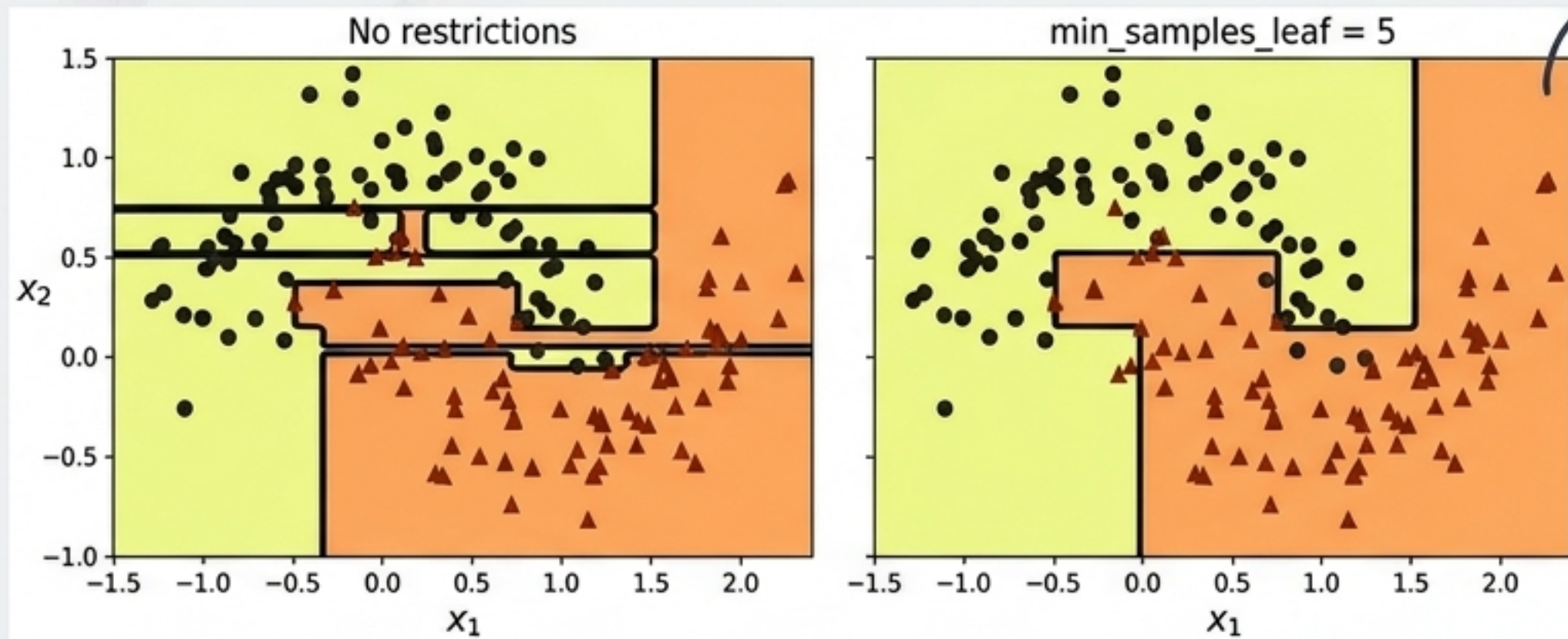
```
import numpy as np
from sklearn.tree import DecisionTreeRegressor

# Tạo dữ liệu mẫu y = x^2 + nhiễu
np.random.seed(42)
X_quad = np.random.rand(200, 1) - 0.5
y_quad = X_quad ** 2 + 0.025 * np.random.randn(200, 1)

# Huấn luyện cây hồi quy (Giới hạn max_depth)
tree_reg = DecisionTreeRegressor(max_depth=2, random_state=42)
tree_reg.fit(X_quad, y_quad)
```

```
def f(x): return x**2 + 0.025 * np.random.randn(1)
tree_reg = DecisionTreeRegressor(max_depth=2, random_state=42)
```

Trực quan hóa Cây Hồi quy



Hình 6-4: Sơ đồ Decision Tree Regression

Giải nghĩa Nút lá:

value: Chính là y trung bình của các mẫu rơi vào lá đó.

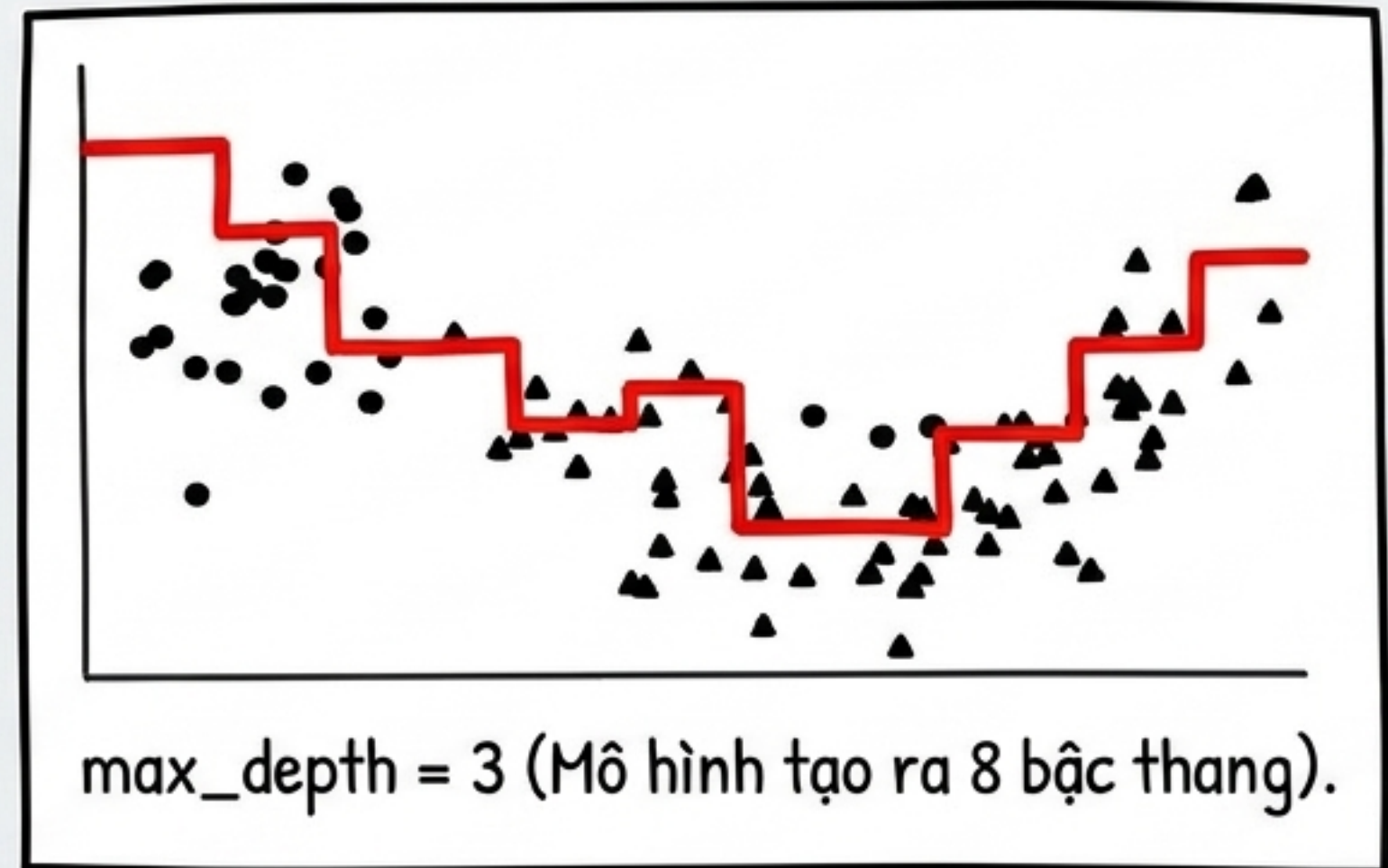
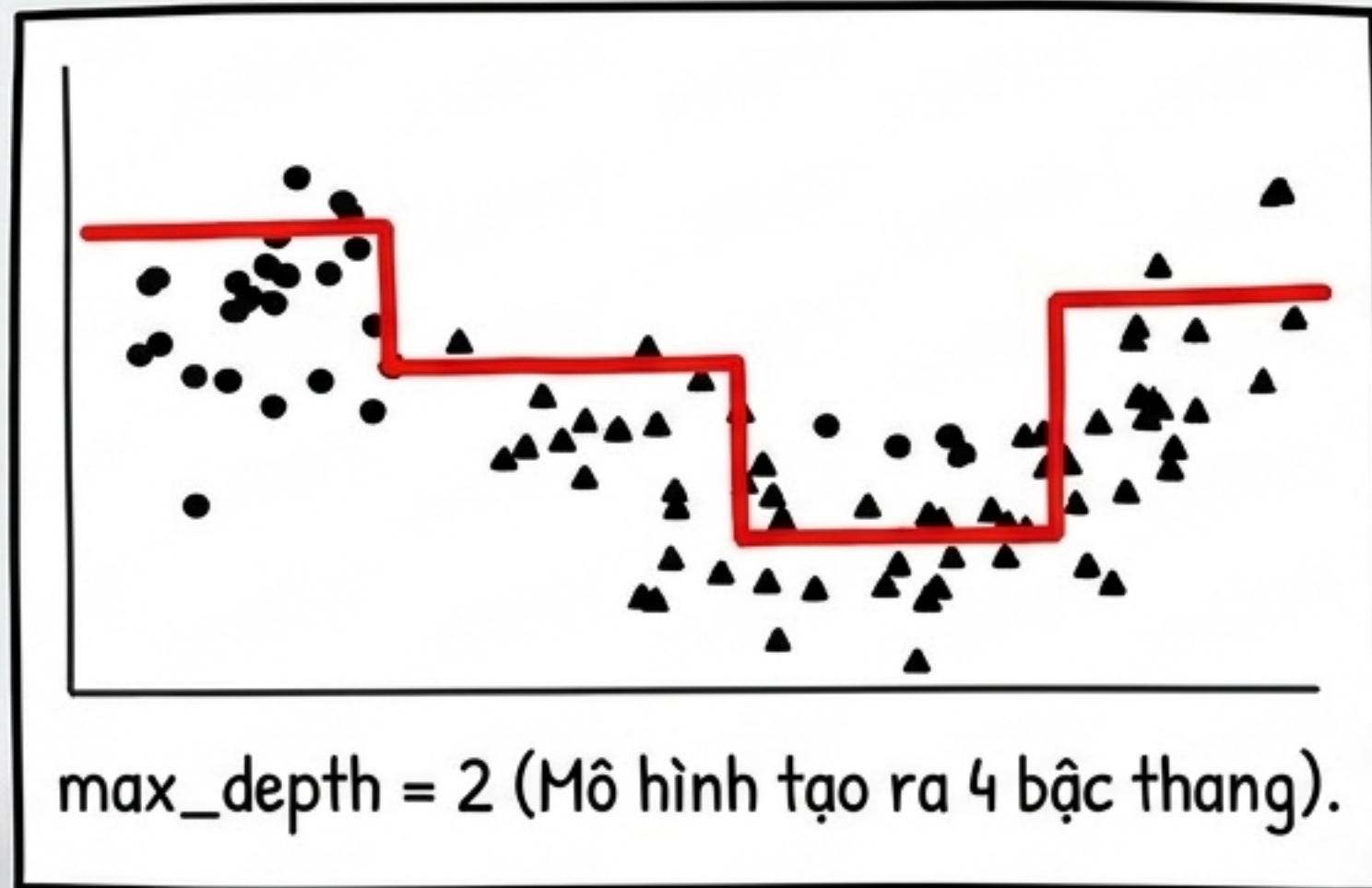
mse: Mean Squared Error (sai số tại nút đó).

Bản chất Dự đoán:

Mang dạng các bậc thang (step-wise) vì trong cùng 1 khoảng chia, mọi giá trị x đều trả ra cùng 1 mức y .

Các Ranh giới Hồi quy (Bậc thang)

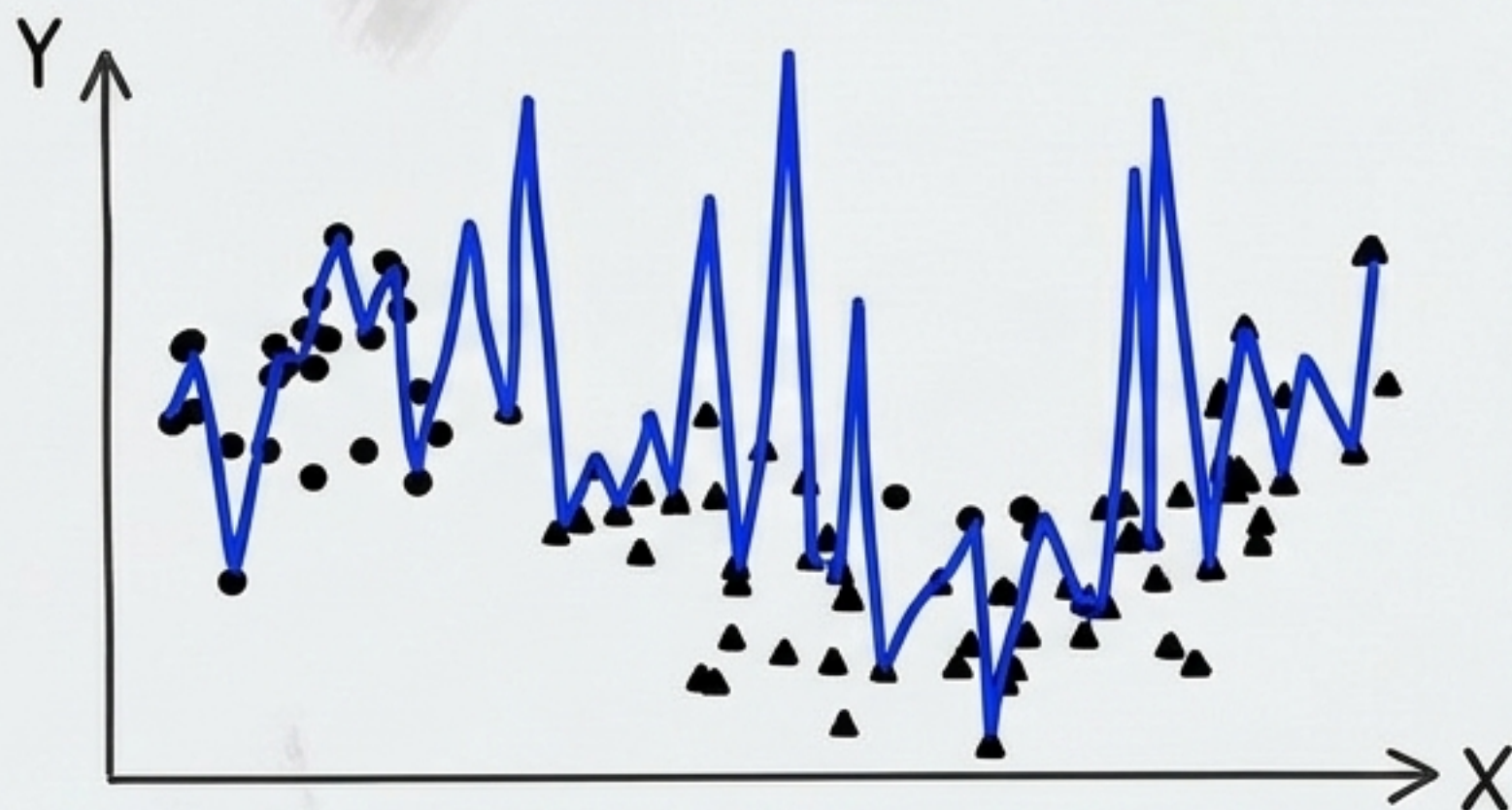
Bản chất: Cây phân chia không gian thành các mảng và dự đoán y là đường thẳng hằng số trên từng mảng.



Hình 6-5: Hàm dự đoán của Cây Hồi quy.

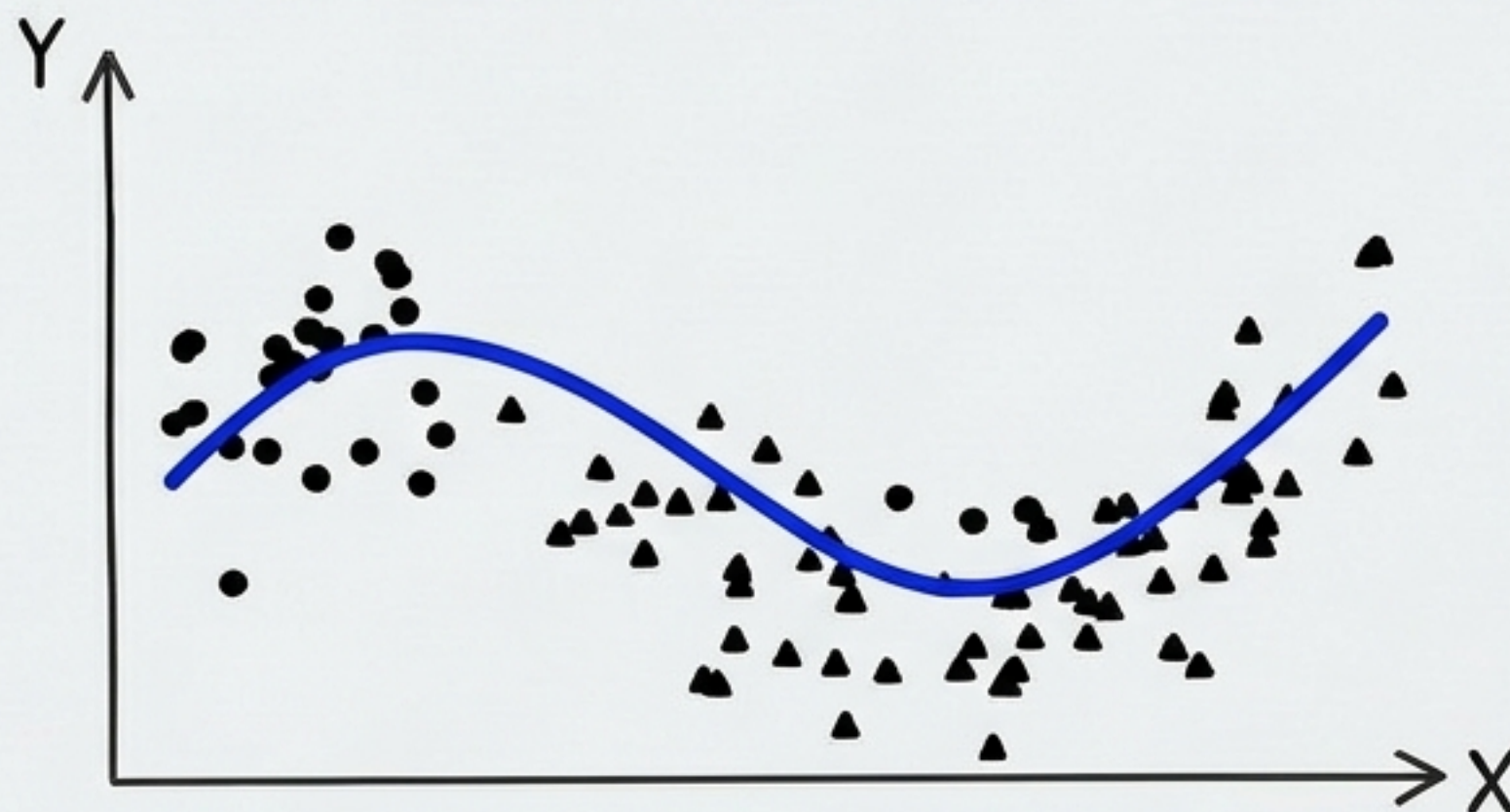
Overfitting trong Hồi quy

Vấn đề (Overfitting):



Cây Hồi quy rất dễ bị Overfitting nếu không ràng buộc. Cây sẽ uốn éo đứt đoạn để đi qua toàn bộ các nhiễu dữ liệu.

Khắc phục (Regularization):



Dùng `min_samples_leaf=10` để ép mô hình mượt mà hơn.

Hàm Chi phí CART cho Hồi quy

Thuật toán CART tối thiểu hóa sai số MSE (thay vì Gini Impurity):

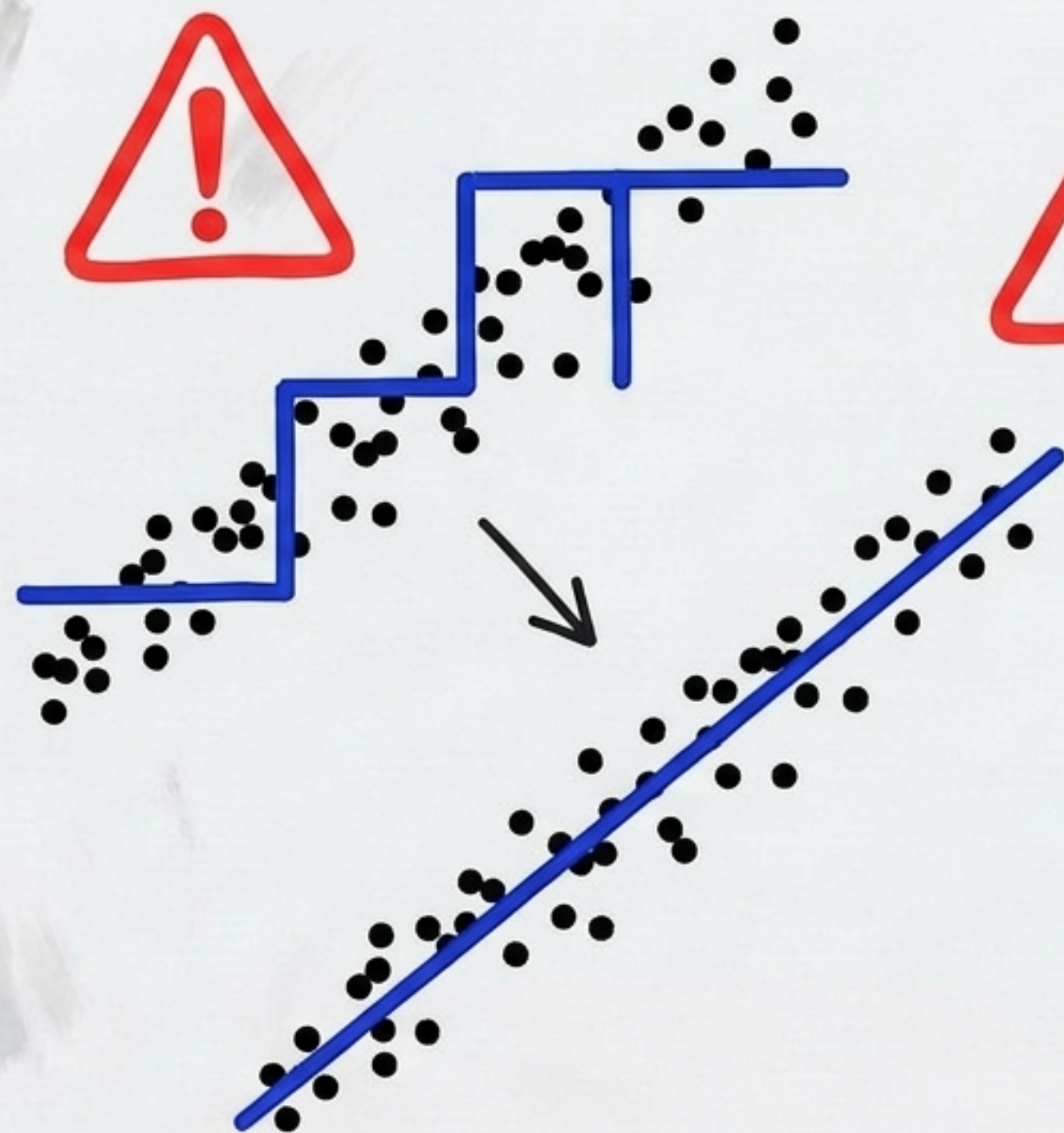
$$J(k, t_k) = \frac{m_{left}}{m} MSE_{left} + \frac{m_{right}}{m} MSE_{right}$$

Trong đó:

$$MSE_{node} = \frac{1}{m_{node}} \sum_{i \in node} (\hat{y}_{node} - y^{(i)})^2$$

Ghi chú: $\hat{y}_{node}$ luôn là Trung bình cộng của các mẫu trong lá.

Hạn chế: Trục chia vuông góc



- Orthogonal splits: Cây quyết định yêu thích sự đơn giản luôn phân chia tập dữ liệu bằng các đường song song hoặc vuông góc với trục tọa độ.
- Hạn chế: Không thể vẽ đường chéo hay đường cong!
- Hệ quả: Dẫn đến sự nhạy cảm cực độ đối với Hướng Xoay của Dữ liệu (Axis Orientation).